

Link Shortener

con Stack di Osservabilità

Flask

MySQL

Prometheus

Grafana

Loki

Ansible

Stefano Videsott · 252739

Ismaele De Giorgi · 237300

Indice

01 Il Progetto

Obiettivi, motivazione, funzionalità

02 Stack & Architettura

Servizi Docker, networking, volumi

03 Osservabilità

Prometheus, Grafana, Loki, Sentry

04 CI/CD & Deployment

GitHub Actions, Ansible

05 Demo Live

Accorciamento link, redirect, dashboard

06 Conclusione

Ringraziamento e discussione

01 · Il Progetto

Il Progetto



Link Shortener

Accorcia URL lunghi in codici brevi a 6 caratteri con TTL configurabile e pannello admin.



Stack di Osservabilità

Metriche, log strutturati e tracciamento errori.



Infrastructure as Code

Deploy riproducibile su Arch Linux con Ansible. CI/CD automatico via GitHub Actions.

Obiettivo del progetto: Il focus non è l'applicazione in sé, ma l'**infrastruttura** che la circonda. Con deployment ripetibile ed automatizzato.

API & Funzionalità

Endpoint Principali

POST /shorten

Crea short link

GET /_error

Triggers an error 500

GET /<short_code>

Redirect

GET /stats/<short_code>

Click, scadenza

GET /health

DB connectivity

GET /metrics

Prometheus

Features

✓ Short code alfanumerico (6 char, $62^6 \approx 56$ miliardi di combinazioni)

✓ Simulazione di errori dell'applicazione lato backend

✓ TTL (default 30 giorni) e cleanup automatico (default 10 min) configurabili

✓ Pannello admin per gestione e monitoraggio link

✓ Log strutturati JSON su ogni richiesta HTTP

02 · Stack & Architettura

Stack Tecnologico

Flask

App Python
+ python-json-logger

MySQL 8

Persistenza
Named Volume Docker

Docker Compose

8 servizi
bridge network

Prometheus

Scrape ogni 15s
5 metriche custom

Grafana

Dashboard
auto-provisioning

Loki + Promtail

Log aggregation
31gg retention

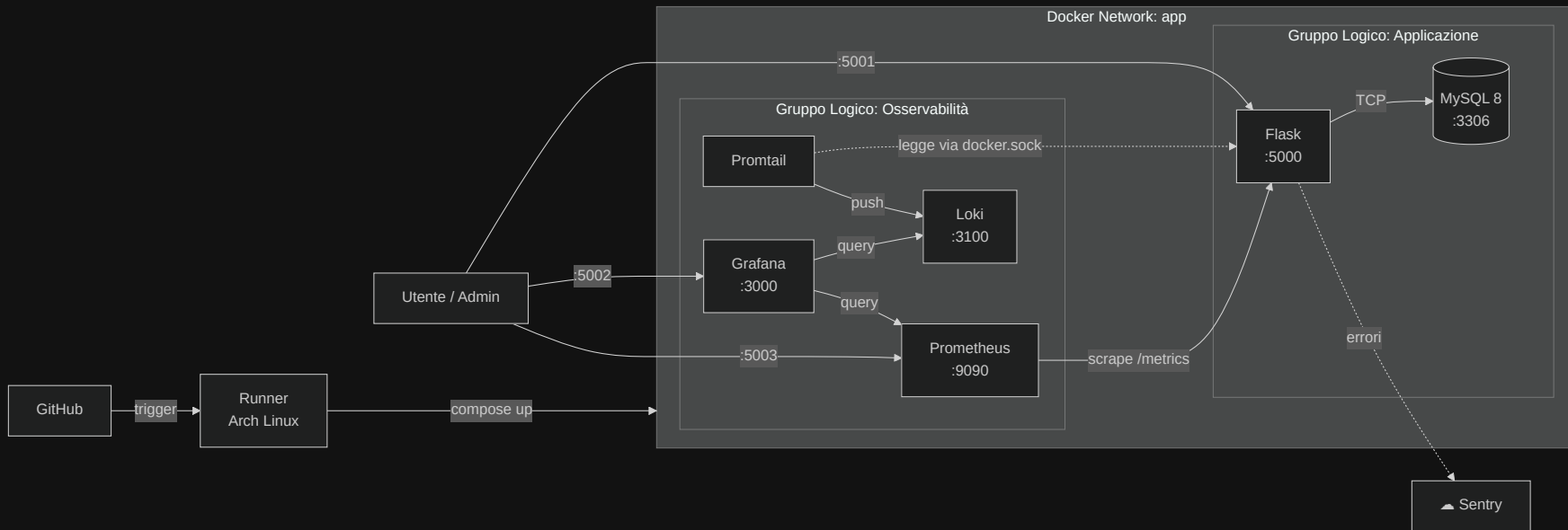
Sentry

Error tracking
cloud + DSN opzionale

Ansible

laC · 3 Play
Arch Linux

Architettura di Sistema



03 · Osservabilità

Segnali impiegati

Metriche

Prometheus + Grafana

Valori numerici aggregati nel tempo. Rispondono a "quanto?" e "quanto spesso?".

Log

Loki + Promtail

Eventi discreti con contesto testuale. Rispondono a "cosa è successo esattamente?".

Errori

Sentry SDK

Eccezioni con stack trace e contesto completo. Rispondono a "perché è andato storto?".

Usati insieme: le metriche identificano **quando** e **quanto** qualcosa va storto — i log spiegano **cosa** è successo esattamente in quel momento.

Metriche — Prometheus

5 Metriche Esposte a `/metrics`

`http_requests_total`

Counter · label: method, endpoint, http_status

`http_request_duration_seconds`

Histogram · label: method, endpoint · P50/P95/P99

`links_created_total`

Counter · link accorciati da avvio

`links_redirected_total`

Counter · redirect effettuati

`last_cleanup_success_timestamp`

Gauge · heartbeat del job di cleanup

Scraping via `prometheus.yml`

```
1  scrape_configs:
2    - job_name: flask
3      static_configs:
4        - targets: ['flask:5000']
5      metrics_path: /metrics
6
7    - job_name: prometheus
8      static_configs:
9        - targets: ['localhost:9090']
10
11    - job_name: grafana
12      static_configs:
13        - targets: ['grafana:3000']
```

Nota: i target usano la porta interna del container (5000 , 3000), non quella esposta sull'host - i servizi comunicano dentro la bridge network.

Dashboard Grafana

Request Rate

req/s per endpoint

```
rate(http_requests_total[1m])
```

Latenza P50/P95/P99

Histogram quantile

```
histogram_quantile(0.99)
```

Tasso Errori 5xx

Error rate % sul totale

```
http_status="5.."
```

Link Creati

Crescita cumulativa

```
links_created_total
```

Redirect

Rate redirect/s

```
links_redirected_total
```

Cleanup Heartbeat

Ultima esecuzione job

```
last_cleanup_success
```

La dashboard è **auto-provisioned** all'avvio tramite `grafana/provisioning/dashboards/` — zero configurazione manuale.

Query PromQL — Traffico & Errori

Richieste al secondo · per endpoint e status

```
rate(http_requests_total[5m])
```

Solo errori 5xx

```
rate(http_requests_total{http_status=~"5.."}[5m])
```

`rate()` calcola la variazione media al secondo nella finestra temporale. Usare `[5m]` per dashboard stabili, `[1m]` per maggiore reattività.

`http_status=~"5.."` usa una regex — `=~` filtra tutti i codici che iniziano con 5 (500, 502, 503 ...).

Query PromQL — Latenza & Business

Percentile 95 — globale

```
histogram_quantile(0.95,  
  sum by (le) (  
    rate(http_request_duration_seconds_bucket[5m])  
  )  
)
```

Redirect rate per link creato

```
rate(links_redirected_total[5m])  
/ rate(links_created_total[5m])
```

Cleanup heartbeat — Deadman Switch

```
time() - last_cleanup_success_timestamp_seconds
```

histogram_quantile(0.95) — il 95% delle richieste è più veloce di questo valore. `le` è il label "less than equal" generato automaticamente dall'Histogram.

Redirect rate — quante volte in media viene usato ogni link creato nell'intervallo. Misura l'utilità reale dei link.

Deadman Switch — se il valore supera **1800s** (30 min) il worker di cleanup si è bloccato silenziosamente. Senza questa metrica l'anomalia sarebbe invisibile.

Log Strutturati

Pipeline di Log

Flask stdout (JSON)

|



Promtail

Docker socket discovery

JSON pipeline stages

Drop /health /metrics 200

|



Loki (31gg retention)

|



Grafana (LogQL)

Log JSON da Flask

```
# python-json-logger - ogni request loggata
_handler.setFormatter(
    JsonFormatter(
        "%(asctime)s %(levelname)s "
        "%(name)s %(message)s"
    )
)
```

Label indicizzato

level → INFO / WARNING / ERROR

Log filtrati: i GET /health 200 e GET /metrics 200 vengono scartati da Promtail (già coperti da Prometheus)

Error Tracking — Sentry

Integrazione Flask

```
_sentry_dsn = os.environ.get("SENTRY_DSN")
if _sentry_dsn:
    import sentry_sdk
    from sentry_sdk.integrations.flask \
        import FlaskIntegration

    sentry_sdk.init(
        dsn=_sentry_dsn,
        integrations=[FlaskIntegration()],
        traces_sample_rate=0.1,
    )
```

Attivo solo se `SENTRY_DSN` è impostato in `.env`.
Disabilitabile senza modificare il codice.

Cattura automatica

Eccezioni non gestite, errori 500, stack trace completi con contesto request

Performance Tracing

10% campionamento transazioni per latenza end-to-end (Flask → MySQL)

Complementare a Loki

Loki = volume di log · Sentry = debug approfondito con contesto variabili

04 · CI/CD & Deployment

GitHub Actions — CI/CD

```
1  name: Deploy
2  on:
3    push:
4      branches: [main]
5    workflow_dispatch:      # trigger manuale
6
7  jobs:
8    deploy:
9      runs-on: self-hosted  # runner sul server Arch
10     timeout-minutes: 15
11
12     steps:
13       - uses: actions/checkout@v4
14
15       - name: Copy .env
16         run: cp /home/deploy/link_shortner-fas/.env .
17
18       - name: Build & Deploy
19         run: |
20           docker compose build
21           docker compose up -d --remove-orphans
22
23       - name: Health check
24         run: curl -sf http://localhost:5001/health
```

Self-Hosted Runner su Arch Linux

● Runner come servizio systemd

Installato come utente `deploy`, già nel gruppo `docker`

● Registrazione via GitHub API

Token ottenuto durante il Play 3 di Ansible

● Nessun SSH in ingresso

Il runner contatta GitHub outbound.

● `actions/checkout@v4` = `git pull`

Fa fetch + reset al commit esatto che ha triggerato il workflow — non serve un `git pull` separato

Ansible — Infrastructure as Code

Play 1 System Setup

- · Installa Docker, Git, UFW
- · Abilita servizio Docker
- · Crea utente `deploy`
- · Autorizza chiave SSH pubblica
- · Firewall: allow 22, 80, 443

Play 2 Deploy App

- · Clone/update repository
- · Crea `.env` da example
- · `Halt` se `.env` non configurato
- · `docker compose pull`
- · `docker compose up -d`

Play 3 CI/CD Runner

- · Scarica runner binary v2.322.0
- · Token via GitHub API (PAT)
- · `config.sh --unattended`
- · `svc.sh install deploy`
- · Avvia servizio systemd

Idempotente

Rieseguibile N volte senza effetti collaterali

Separazione

3 Play con responsabilità distinte e ordinamento preciso

Zero Secrets

`vars.yml` `gitignore` · `.env` gestito sul server

05 · Demo Live

Demo

1. Applicazione

- → Accorciare un link
- → Testare il redirect
- → Vedere le statistiche
- → Pannello admin

2. Grafana Dashboard

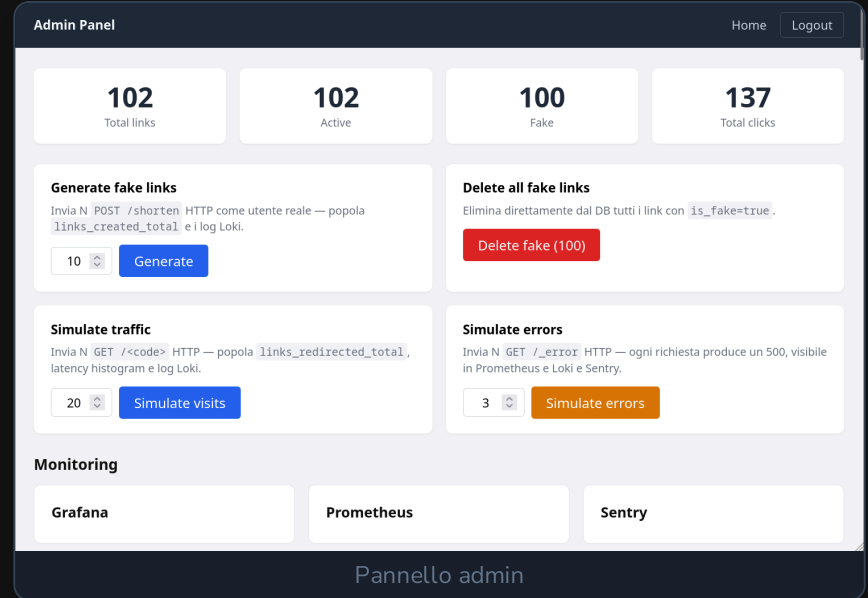
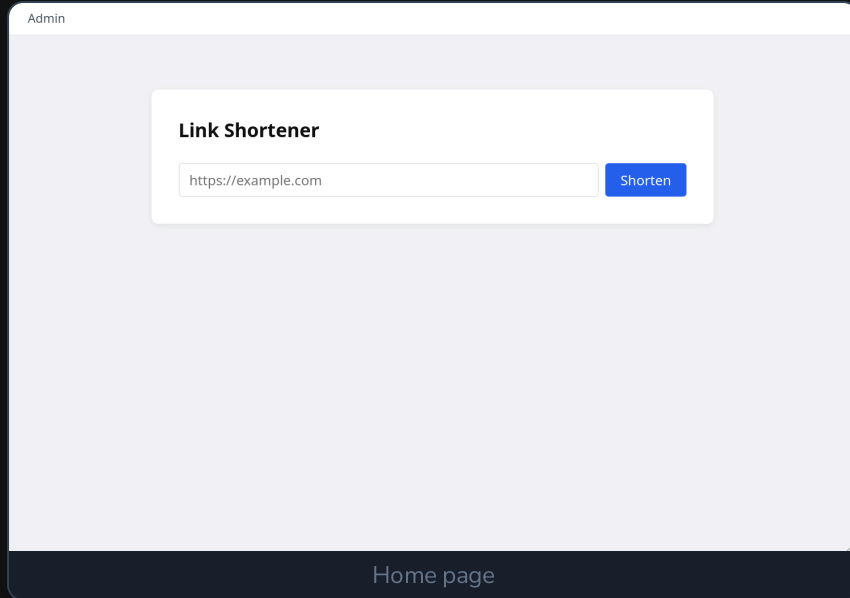
- → Request rate live
- → Latenza P95/P99
- → Log in tempo reale
- → Cleanup heartbeat

3. CI/CD

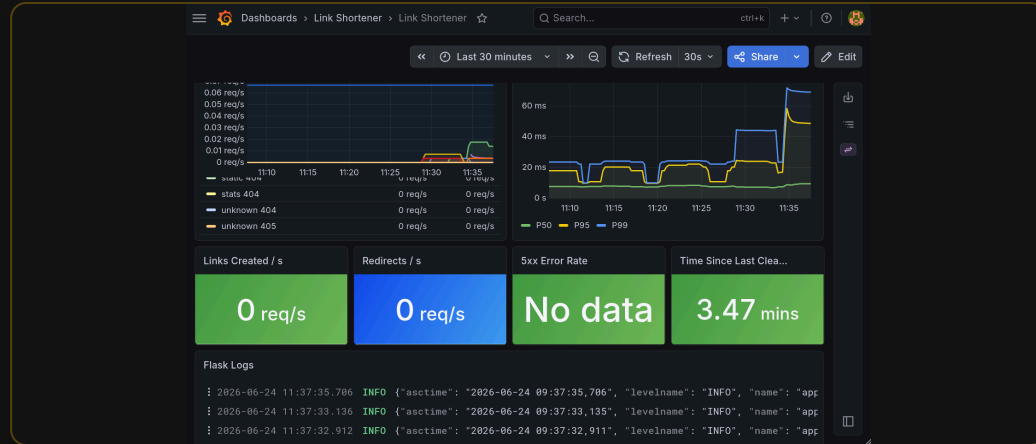
- → Push su main
- → Workflow GitHub
- → Runner sul server
- → Deploy automatico

App :5001 · Grafana :5002 · Prometheus :5003

Screenshot · Applicazione



Screenshot · Dashboard Grafana



Grazie per l'attenzione

App

<https://shortner.stefanovidesott.com>

Grafana

[https://shortner-
grafana.stefanovidesott.com](https://shortner-grafana.stefanovidesott.com)

Prometheus

[https://shortner-
prometheus.stefanovidesott.com](https://shortner-prometheus.stefanovidesott.com)